

```

File Edit Format Run Options Window Help
import face_recognition
import cv2
import numpy as np
import time
from espeak import espeak
previous = "unknown"
# This is a demo of running face recognition on live video from your webcam. It's
# other example, but it includes some basic performance tweaks to make things run
# 1. Process each video frame at 1/4 resolution (though still display it at full
# 2. Only detect faces in every other frame of video.

# PLEASE NOTE: This example requires OpenCV (the "cv2" library) to be installed
# OpenCV is "not" required to use the face_recognition library. It's only required
# specific demo. If you have trouble installing it, try any of the other demos that
# don't require it.

# Get a reference to webcam #0 (the default one)
video_capture = cv2.VideoCapture(0)

# Load a sample picture and learn how to recognize it.
ash_image = face_recognition.load_image_file("ash.jpg")
ash_face_encoding = face_recognition.face_encodings(ash_image)[0]

# Load a second sample picture and learn how to recognize it.
alhil_image = face_recognition.load_image_file("Alhil.jpg")
alhil_face_encoding = face_recognition.face_encodings(alhil_image)[0]

# Load a sample picture and learn how to recognize it.
atul_image = face_recognition.load_image_file("Atul.jpg")
atul_face_encoding = face_recognition.face_encodings(atul_image)[0]

# Load a second sample picture and learn how to recognize it.
gaurav_image = face_recognition.load_image_file("Gaurav.jpg")
gaurav_face_encoding = face_recognition.face_encodings(gaurav_image)[0]

```

Figure 12: Face recognition code

```

face_encodings = []
face_names = []
process_this_frame = True

while True:
    # Grab a single frame of video
    ret, frame = video_capture.read()

    # Resize frame of video to 1/4 size for faster face recognition processing
    small_frame = cv2.resize(frame, (0, 0), fx=0.25, fy=0.25)

    # Convert the image from BGR color (which OpenCV uses) to RGB color (which face_recognition uses)
    rgb_small_frame = small_frame[:, :, ::-1]

    # Only process every other frame of video to save time
    if process_this_frame:
        # Find all the faces and face encodings in the current frame of video
        face_locations = face_recognition.face_locations(rgb_small_frame)
        face_encodings = face_recognition.face_encodings(rgb_small_frame, face_locations)

        face_names = []
        for face_encoding in face_encodings:
            # See if the face is a match for the known face(s)
            matches = face_recognition.compare_faces(known_face_encodings, face_encoding)
            name = "Unknown"

            # If a match was found in known_face_encodings, just use the first
            # if True in matches:
            #     first_match_index = matches.index(True)
            #     name = known_face_names[first_match_index]

            # Or instead, use the known face with the smallest distance to the current face
            face_distances = face_recognition.face_distance(known_face_encodings, face_encoding)
            best_match_index = np.argmin(face_distances)
            if matches[best_match_index]:
                name = known_face_names[best_match_index]
                if previous != name:
                    previous = name
                    print(name)
                    #espeak.synth("Hey I Can Identify your face")
                    #espeak.synth("Your name is")
                    #espeak.synth(name)
                    espeak.set_voice("En")
                    espeak.set_voice("Whisper")
                    espeak.synth("Hey hello ")
                    espeak.synth(name)
                    espeak.synth("How are you ")
                    espeak.synth("have a nice day")

            else:
                print("old same")

        face_names.append(name)

    process_this_frame = not process_this_frame

# Display the results
for (top, right, bottom, left), name in zip(face_locations, face_names):
    # Scale back up face locations since the frame we detected in was scaled
    top *= 4
    right *= 4
    bottom *= 4
    left *= 4

```

Figure 14: Open Cv processing image

```

File Edit Format Run Options Window Help

rahul_face_encoding,
shipra_face_encoding,
vinay_face_encoding,
vinita_face_encoding,
parul_face_encoding,
tanyaaneja_face_encoding,
deba_face_encoding,
tanya_face_encoding,
reshu_face_encoding,
astha_face_encoding,
akhilesh_face_encoding,

]
known_face_names = [
    "Ashwini kumar",
    "Akhil ",
    "Atul",
    "gaurav Lo",
    "mukul",
    "shavita",
    "rahul",
    "shipra",
    "vinay",
    "vinita",
    "parul",
    "tanya ",
    "deba",
    "tanya",
    "reshu",
    "astha",
    "akhilesh"
]

```

Figure 13: Database of known face names

```

File Edit Format Run Options Window Help

# name = known_face_names[first_match_index]

# Or instead, use the known face with the smallest distance to the current face
face_distances = face_recognition.face_distance(known_face_encodings, face_encoding)
best_match_index = np.argmin(face_distances)
if matches[best_match_index]:
    name = known_face_names[best_match_index]
    if previous != name:
        previous = name
        print(name)
        #espeak.synth("Hey I Can Identify your face")
        #espeak.synth("Your name is")
        #espeak.synth(name)
        espeak.set_voice("En")
        espeak.set_voice("Whisper")
        espeak.synth("Hey hello ")
        espeak.synth(name)
        espeak.synth("How are you ")
        espeak.synth("have a nice day")
    else:
        print("old same")

face_names.append(name)

process_this_frame = not process_this_frame

# Display the results
for (top, right, bottom, left), name in zip(face_locations, face_names):
    # Scale back up face locations since the frame we detected in was scaled
    top *= 4
    right *= 4
    bottom *= 4
    left *= 4

```

Figure 15: Greeting code part to call and say when person is detected